

INDIVIDUAL DEVELOPMENT QUESTIONNAIRE

CTMOBPGL Final Project — To be answered individually and submitted with the project

Name: Gomez, Rhianna Mikaela R.

Group Members: Adraneda, Ann Nicole P.

Section: ITE302

Tan, Lorenzo Carlos R.

Date: May 28, 2026

1. Describe the overall architecture of your Flutter app. How did you organize your files and folders, and why did you choose that structure?

Think about: lib/ folder structure, separation of screens, models, widgets, and services.

Our app uses a **layered folder structure** inside the `lib/` folder. We separated the files by their purpose — `core/` holds the app's theme and navigation setup, `models/` holds the data classes, `screens/` contains the UI for each feature, `providers/` manages the app's state, `services/` handles Firebase and API calls, and `widgets/` stores reusable UI components. We organized it this way so each file has a clear role.

2. What state management approach did you use (e.g., `setState`, `Provider`, `Riverpod`)? Explain why you chose it and give one specific example of how you applied it in your app.

Think about: a widget or screen where state needed to be shared or updated reactively.

We used **Provider** for state management because it allows the app to automatically update the UI whenever data changes, without manually refreshing the screen. For example, in my Timetable screen, when a subject is added or deleted, the list updates on its own because the screen is wrapped in a `Consumer<SubjectProvider>` that listens for changes. I also used `setState()` for simpler local changes — like when the user long-presses a card to enter select mode, only that screen needs to update, so `setState()` is enough for that case.

3. How does your app handle data persistence? Walk through the process of saving and retrieving data — from the user action to the storage layer.

Think about: the flow from a button tap → function call → database/API → UI update.

We used **Cloud Firestore** to save and retrieve data. For example, in my Tasks module, when the user fills out the form and taps the save button, the app first validates the inputs, then saves the data to Firestore under a `tasks` collection. Firestore automatically syncs the data in real-time, so the task list on screen updates instantly without needing to manually reload the page. When the app opens again, it fetches only the current user's data by filtering with their `userId`. I also added a local notification that reminds the user 24 hours before a task is due, which is handled by the `flutter_local_notifications` package.

4. Describe a specific bug, error, or challenge you personally encountered while building the app. What was the problem, how did you debug it, and what was your solution?

Think about: a Flutter widget error, a Dart runtime error, an async timing problem, or a layout bug.

One problem I ran into was that the save buttons on the Add Subject, Add Task, and Add Exam forms were not doing anything when tapped — no error, just nothing happened. To find the cause, I added `debugPrint` statements in different parts of the code to trace where it was stopping. I discovered that the problem was with the `userId` — it was coming out empty because the form was opened inside a bottom sheet, which had lost access to the app's Provider data. So when it tried to get the current user's ID, it returned null and the save process silently stopped. I fixed it by getting the `userId` before opening the bottom sheet and passing it directly into the form as a parameter, so it always had the correct value no matter where the form was opened from.

5. If you were to rebuild or improve this app with more time, what specific Flutter feature, package, or architectural change would you add, and why?

Think about: animations, better navigation, offline support, testing, performance optimization, etc.

If I had more time, I would add an **Add Friend feature** where students can connect with their classmates inside the app. With this, users could share their timetable, exams or tasks with friends, which would be helpful especially for groupmates who have the same class schedule. This would make the app more social and useful compared to just being a personal planner.